

Autómatas, Teoría de Lenguajes y Compiladores

Diseño e implementación de un lenguaje

Especificación

Teo Fumagalli

Ingeniería Informática, ITBA

Buenos Aires, Argentina

September 2, 2026

Contents

1	Equipo	2
2	Repositorio	2
3	Dominio	2
4	Construcciones	2
4.1	Variables	3
4.2	Tipos de datos	3
4.3	Funciones y componentes	4
4.4	Composición de funciones	5
4.5	Expresiones	6
4.6	Parámetros de compilación y recursos externos	7
4.7	Layout	8
4.8	Templates	8
5	Casos de prueba	9
6	Ejemplos	10
6.1	Página simple	10
6.2	Tabla de CSV	11
6.3	Tabla de CSV genérica	12

1 Equipo

Nombre	Apellido	Legajo	E-mail
Teo	Fumagalli	66048	tfumagalli@itba.edu.ar

2 Repositorio

La solución será versionada en <https://github.com/teofum/tpe-tla>

3 Dominio

Desarrollar un lenguaje para generación de sitios web estáticos (HTML + CSS + JS) que puedan ser hosteados por un servidor web estándar (e.g. Apache, nginx). El lenguaje debe permitir crear programas cuya salida sea una página HTML válida, crear componentes reutilizables en la forma de funciones, programar lógica sencilla (condicionales, iterar listas), y leer datos de entrada de ciertos formatos (JSON, CSV) a variables. También deberá permitir generar páginas con un formato predefinido a partir de archivos CSV, JSON y Markdown.

Con tal de reducir la complejidad del proyecto, los estilos (CSS) y cualquier comportamiento frontend (JS) serán importados como *assets* externos y referenciados en el HTML generado.

Toda la evaluación del programa que genera una página HTML será realizada dentro del compilador como parte del proceso de compilación. El artefacto de entrada es uno o más archivos fuente o un directorio con archivos fuente y recursos (CSS, JS, imágenes, datos en formato JSON o CSV, etc.), y el artefacto de salida es un directorio con archivos HTML correspondiendo a cada archivo fuente que retorne una página completa, y todos los recursos necesarios.

La implementación satisfactoria del lenguaje permitiría escribir sitios web sencillos (estáticos) con un *toolchain* muy simple y sin dependencias externas (e.g. npm), ningún componente requerido en tiempo de ejecución, y en un lenguaje más amigable que HTML estándar.

4 Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

4.1 Variables

El lenguaje permitirá definir variables con un *scope* localizado al bloque más cercano (con reglas similares a e.g. C). Cada variable tiene un tipo y un valor; las variables deberán ser inicializadas explícitamente al definir las y no hacerlo será un error.

También se podrán definir constantes, con semántica idéntica a una variable con la excepción de que no pueden ser modificadas una vez definidas. Se permitirá la definición de variables con el mismo nombre que otras (*shadowing*), pero nunca dentro del mismo *scope*.

Ejemplos de sintaxis

```
x: int = 5           // Declara la variable "x" con tipo "int" y valor 5
y := 5              // Se infiere el tipo "int" del literal 5
z := "Hello, world!" // Se infiere el tipo "string" del literal
z_2 := z            // Inicializar con otra variable toma su tipo de datos
x_2: int = z         // ERROR: z es de tipo "string"
```

4.2 Tipos de datos

El sistema de tipado será **estático fuerte**: los tipos de datos serán verificados en tiempo de compilación, y el tipo de datos de una variable será permanente durante toda la vida de esa variable.

El lenguaje contará con tipos de datos generales, tanto primitivos como compuestos, y otros específicos al dominio. Los tipos de datos serán:

Tipos primitivos: `int` (números enteros); `float` (números decimales, siguiendo IEEE 754); `bool` (verdadero/falso); `string` (cadenas de texto UTF-8); `nil` (tipo vacío, su único valor válido es `nil`).

Tipos de dominio: `element`, representa un elemento HTML con su nombre de *tag*, atributos, y descendientes. Un archivo fuente que retorna una variable de tipo `element` es considerado un punto de entrada, y genera una página en la salida.

Estructuras: definidas con la palabra reservada `struct`, son una agrupación de otros valores cada uno con un nombre y tipo de datos.

Uniones: definidas con la palabra reservada `union`, representan un tipo que puede tomar uno de varios tipos de datos, junto con información del tipo actual de la variable (*tagged union*).

Opcionales: definidos a partir de un tipo `T` como `T?`, es un atajo para el patrón común `union { T, nil }`.

Enumeraciones: definidas con la palabra reservada `enum`, son un tipo de datos que toma valores específicos de una lista.

Listas: definidas a partir de un tipo `T` como `[]T` ("lista de `T`"). Tamaño dinámico.

Tuplas: definidas a partir de tipos `T`, `U`, ... como `<T, U, ...>`. Tamaño fijo, con elementos de tipos heterogéneos.

Mapas: definidos a partir de dos tipos `K`, `V` como `[K]V` ("mapa de `V` indexado por `K`"). Sintaxis especial para el caso `K = string`, ya que es un uso muy común (al trabajar con JSON).

Funciones: definidas a partir de sus tipos de parámetros y de retorno, con la palabra reservada `function`.

Los tipos de datos de usuario se definen con el operador `is`:

```
# Struct usuario con campos int, string y opcional(string)
User is struct {
    id: int,
    username, first_name, last_name: string,
    profile_image_url: string?,
}

# Lista de elementos que pueden tomar uno de cuatro tipos
Article is []union { Paragraph, Image, Quote, CodeBlock }
```

4.3 Funciones y componentes

Las funciones se definen como un tipo de datos de usuario, con la adición de un cuerpo de función que contiene su código:

```
display_name is function(user: User) -> string {
    concat(" ", user.first_name, user.last_name)
}
```

Toda función debe tener un tipo de retorno, indicado por `->` siguiendo su lista de parámetros. Los argumentos de una función se pasan por valor, y no pueden ser modificados.

Un componente es sencillamente una función que retorna el tipo `element`:

```
user_avatar is function(user: User) -> element {
    div.flex.flex-row.gap-4.items-center {
        // ...
    }
}
```

La sintaxis de llamada de función es similar a un lenguaje como C, con la diferencia que si una función no toma parámetros, o todos sus parámetros tienen un valor por defecto y no se pasa ninguno al llamarla, se puede omitir la lista de parámetros por completo.

```
hello is function(name: string = "world") -> string {  
    "Hello, ${name}!"  
}
```

```
name := display_name(user)  
str1 := hello(name)           // "Hello, <users's name>!"  
str2 := hello                 // "Hello, world!"
```

Los componentes se definen como cualquier función, pero tienen comportamientos especiales al llamarlos: se admiten cadenas de texto arbitrarias (sin espacios) prefijadas con `.` (cualquier cantidad) o `#` (sólo una) entre el nombre de función y la lista de parámetros. Las primeras se pasan como `[]string` al parámetro especial `class`, y la segunda como `string` al parámetro especial `id`. En caso de no existir estos parámetros, el uso de su sintaxis especial es un error.

La sintaxis de llamada especial para componentes existe para facilitar la escritura de código markup:

```
// Sin sintaxis especial de llamada  
div(class = "flex flex-row gap-4 items-center", id = "myDiv")
```

```
// Con sintaxis especial de llamada  
div.flex.flex-row.gap-4.items-center#myDiv
```

4.4 Composición de funciones

La construcción que permite utilizar el lenguaje para construir estructuras HTML complejas de modo sencillo es la **composición de funciones**. Esto consiste en pasar una expresión arbitraria como parte de una llamada a función. La expresión es tratada como un parámetro especial, y es inserta en cada lugar donde la función tenga la palabra reservada `compose`:

```
twice is function -> <string, string> {  
    compose  
    compose  
}  
  
foo := twice "Hello"           // <"Hello", "Hello">  
bar := twice {
```

```
    who := "world"
    "Hello ${who}!"
}                                     // <"Hello, world!", "Hello, world!">
```

Un bloque de código pasado para composición captura el *scope* de la llamada a función; es decir, tiene acceso a las variables del lugar donde se llama. **No tiene acceso al *scope* interno de la función.**

4.5 Expresiones

El lenguaje será **basado en expresiones**, es decir, todo en el lenguaje es una expresión que retorna un valor excepto las definiciones de tipo de datos.

1. Una asignación retorna el valor asignado a la variable; esto permite encadenar asignaciones. Las asignaciones tienen comportamiento especial.
2. Una llamada a función retorna el valor de retorno de la función.
3. Una expresión condicional retorna el valor de la rama que haya sido tomada en base a la o las condiciones. Todas deben retornar el mismo tipo.
4. Un *loop* retorna un valor de tipo `[]T`, donde `T` es el tipo que retorna la expresión interna.

Un bloque también es una expresión, que consiste de un agrupamiento lineal y ordenado de otras expresiones. El tipo de retorno de un bloque se decide según su contexto:

- Si el bloque es el cuerpo de una función, su tipo de retorno coincide con el de la función.
- En cualquier otro caso, el tipo de retorno se infiere del valor, que sigue las reglas detalladas a continuación.

El valor de retorno de un bloque está dado por las siguientes reglas:

1. Toda expresión dentro del bloque, **excepto las asignaciones**, agrega su valor de retorno al del bloque.
2. Si el bloque contiene una única expresión que no es una asignación, retorna directamente su valor.
3. Si el bloque contiene varias expresiones y todas retornan un mismo tipo `T`, entonces retorna una lista de tipo `[]T` con los valores de todas sus expresiones (que no sean asignaciones).

4. Si el bloque contiene varias expresiones y retornan distintos tipos, entonces retorna una tupla con los valores de todas sus expresiones (excepto asignaciones).

En el caso del cuerpo de una función, si el valor de retorno no coincide con el tipo declarado en la función, es un error, excepto en el caso que el bloque retorne `[]T` y la función tenga como tipo de retorno una tupla de `T` con el tamaño apropiado.

Ejemplos

```
// Retorna string ("Hello, world!")
{
  greeting := "Hello, "
  greeting = greeting + "world!"
  greeting
}

// Retorna []int ([42, 505, 77])
{
  42
  500 + 5
  77
}

// Retorna <int, string> (<42, "Hello, world!">)
{
  (80 + 4) / 2
  who := "world"
  "Hello, ${who}!"
}
```

4.6 Parámetros de compilación y recursos externos

El lenguaje tendrá la posibilidad de definir variables cuyo valor se toma como parámetro en tiempo de compilación mediante la palabra reservada `var`:

```
// Al invocar el compiler se pasa --var title:"Book Inventory"
page_title := var title default "Data Table"

// Al invocar el compiler se pasa --var src:"books.csv"
// No tiene default, si no se le da un valor es error de compilación
csv_source := var csv
```

También se dará la posibilidad de incluir recursos externos mediante la palabra reservada `import`, con distintos usos dependiendo del tipo de recurso:

import url retorna la url del recurso en el artefacto de salida. Útil para recursos estáticos como imágenes o CSS.

import csv parsea un archivo CSV y retorna una tupla de headers y filas de datos. Opcionalmente se le puede pasar un tipo de datos struct con la expresión **import csv of T**, en cuyo caso retorna una lista de ese tipo para las filas de datos y valida la cantidad de campos.

import json parsea un archivo JSON y retorna una estructura de valor JSON, que consiste en una unión de todos los tipos que soporta el formato. Opcionalmente se le puede pasar un tipo de datos con la expresión **import json of T**, en cuyo caso retorna T y valida que el JSON pueda adaptarse a ese tipo.

import md parsea un archivo Markdown y retorna una lista `[]element` con los elementos del archivo. Opcionalmente se le puede pasar una lista de componentes con la expresión **import md with components**, donde **components** es un mapa con claves `MdComponent` (enumeración *builtin*) y valores función que aceptan composición y retornan `element`.

4.7 Layout

En sitios complejos con varias páginas, frecuentemente es deseable repetir una gran parte del layout de cada página variando únicamente el contenido. El lenguaje cubrirá este caso de uso mediante la funcionalidad de *layout*.

Un *layout* es una página especial, definida en un archivo dentro de un directorio fuente. Se comporta como un componente especial: cuando un directorio contiene un *layout*, las páginas en ese directorio y todos sus subdirectorios son automáticamente "envueltas" en el mismo. Esto permite contener un *layout* dentro de otro.

4.8 Templates

En algunos casos puede ser deseable tener el contenido del sitio en formatos simples (e.g. Markdown para un blog, CSV para tablas de datos) y generar automáticamente páginas para cada archivo. El lenguaje cubrirá esta necesidad mediante la funcionalidad de *templates*.

Un template es una página especial, definida en un archivo `template.xx` donde `xx` es el tipo de archivo al cual aplica. Dentro de un template, se define la variable global `content_path` la cual contiene el path al archivo fuente. Esta variable se puede combinar con `import`:

```
// Ejemplo CSV
headers, rows := import csv content_path

// Ejemplo MD
article := import md with components content_path
```

Al encontrarse un archivo de un tipo soportado, el compilador deberá buscar el template apropiado más cercano entre el directorio del archivo y la raíz del directorio fuente. Si se encuentra un template, se compila una copia del template en lugar del archivo, pasando su path en la variable `content_path`.

5 Casos de prueba

Se proponen los siguientes casos de prueba de **aceptación**:

1. Un programa que genere una página simple con texto.
2. Un programa que genere una página simple incluyendo CSS externo.
3. Un programa que genere múltiples páginas en un directorio.
4. Un programa que defina un componente y lo utilice en una página.
5. Un programa que defina un componente que acepta composición.
6. Un programa que genere una página en base a datos de un CSV.
7. Un programa que genere una página en base a datos de un JSON.
8. Un programa que importe un archivo Markdown.
9. Un programa que defina un *layout* y lo utilice en varias páginas.
10. Un programa que defina un *template* y lo utilice con una o varias fuentes Markdown.

Así como los siguientes casos de prueba de **rechazo**:

1. Un programa malformado.
2. Un programa que intente operar con tipos de datos incompatibles.
3. Un programa que asigne una variable que no se utiliza.
4. Un programa que defina una función que no se utiliza.
5. Un programa que defina un tipo de datos que no se utiliza.

6 Ejemplos

6.1 Página simple

Este ejemplo muestra la creación de una página sencilla, utilizando recursos externos como imágenes y CSS.

```
// "import url" toma un path a recursos en el directorio fuente.
// En compilación, se transforma en url relativa de salida.
my_cool_image := import url "img/cool.png"
tailwind_css  := import url "css/tailwind.css"

html {
  head {
    title "My cool website"
    stylesheet(tailwind_css)
  }
  body.p-8.pb-16 {
    main.max-w-2xl.mx-auto {
      h1.text-2xl.font-bold.mb-4 "Hello, world!"
      p.text-sm.text-black/80.mb-2 {
        "This is some text"
        "and this is some more text"
      }
      img.border.rounded-md(src = my_cool_image)
    }
  }
}
```

6.2 Tabla de CSV

Este ejemplo muestra la creación de una tabla con datos cargados de un CSV. Demuestra el uso de `import csv`, structs, y estructuras de control de flujo como *loops* y condicionales.

```
Book is struct {
  title: string,
  isbn: string,
  price: float,
  stock: int,
  amount_sold: int,
}

// "import csv" toma un path a recursos en el directorio fuente.
// En compilación, parsea el CSV y retorna los datos con el tipo dado, causando
// un error si los tipos no coinciden.
// En este caso retorna una tupla <[string]string, []Book>
headers, books := import csv of Book "data/csv/books.csv"
tailwind_css := import url "css/tailwind.css"

html {
  head {
    title "Book inventory"
    stylesheet(tailwind_css)
  }
  body.p-8.pb-16 {
    main.max-w-2xl.mx-auto {
      h1.text-2xl.font-bold.mb-4 "Book Inventory"

      table {
        thead {
          tr.font-bold {
            th.py-1.px-2 headers.title
            th.py-1.px-2 headers.isbn
            th.py-1.px-2 headers.price
            th.py-1.px-2 headers.stock
            th.py-1.px-2 headers.amount_sold
          }
        }
        tbody {
          for book in books {
            tr {
              th.font-semibold.font-italic {
                a.text-underline(href = "/books/${book.isbn}") book.title
              }
              td book.isbn
              td format("${%.2f}", book.price)
              td {
                if book.stock == 0 {
                  "Out of stock"
                } else {
                  book.stock "units"
                }
              }
              td { book.amount_sold "units" }
            }
          }
        }
      }
    }
  }
}
```

6.3 Tabla de CSV genérica

Este ejemplo muestra la definición de un componente para dibujar tablas a partir de un conjunto de datos arbitrario, y su uso para dibujar una tabla con datos de un CSV de formato desconocido. Demuestra el uso de componentes, mapas, y de `import csv` sin tipo de datos definido.

```

csv_table is function(headers: [string]string, data: [][string]string) -> element {
  fields := for key, header in headers key

  table {
    thead {
      tr.font-bold {
        for key, header in headers th.py-1.px-2 header
      }
    }
    tbody {
      for item in data {
        tr {
          th.font-semibold item[fields[0]]
          for field_name in fields[1:] {
            td item[field_name]
          }
        }
      }
    }
  }
}

// Al no pasarle un tipo de datos, "import csv" retorna una tupla
// <[string]string, [][string]string>, donde las claves de los mapas son
// generadas a partir de los headers.
headers, books := import csv "data/csv/books.csv"
tailwind_css    := import url "css/tailwind.css"

html {
  head {
    title "Book inventory"
    stylesheet(tailwind_css)
  }
  body.p-8.pb-16 {
    main.max-w-2xl.mx-auto {
      h1.text-2xl.font-bold.mb-4 "Book Inventory"

      csv_table(headers, books)
    }
  }
}

```